

IEEE 14-Bus Battery Siting — Model Reference

Contents

IEEE 14-Bus Battery Siting — Model Reference	1
1 Use Case Overview	2
2 Network: IEEE 14-Bus	3
2.1 Bus Data	3
2.2 Branch Data	3
3 Generators	3
4 Load Profile	4
5 Battery Specifications	4
6 Problem Formulation	4
6.1 Objective	4
6.2 Decision Variables	5
6.3 Constraints	5
6.4 Siting Algorithm	5
7 Comparison	6

1 Use Case Overview

- **Network:** IEEE 14-bus, 20 branches, 5 generators (MATPOWER case14)
- **New load:** 200 MW flat AI datacenter added at Bus 4
- **Batteries:** 4 units (50 MW / 200 MWh each), placement to be optimised
- **Search space:** $C(14,4) = 1,001$ candidate placements
- **Objective:** minimise total 24-hour system dispatch cost
- **Horizon:** single 24-hour day, hourly resolution
- **Power flow:** DC approximation (PTDF-based, lossless, no reactive power)
- **Source:** <https://github.com/Square-Peg-Technologies/square-peg-quantum-challenge>

Key Files and Variables

File	Variable / Function	Description
ieee14.py	Case Case.factors CaseDescription.fbar CaseDescription.branch CaseDescription.bus	Grid object: PTDF, fbar, power_demand 24-hour demand multipliers (list of 24) Branch flow limits (MW), 20 elements Branch r, x, b, ratio data Bus Pd, Qd base values
assets.py	GENERATORS BATTERIES DATACENTER_BUS DATACENTER_MW	List of 5 generator dicts (bus, Pmin, Pmax, cost) List of 4 battery dicts (power_mw, capacity_mwh, efficiency, init_soc) None (base case, no datacenter) 0.0 (base case)
assets_dc_bus4.py	DATACENTER_BUS DATACENTER_MW	4 (active scenario) 200.0
locations.py	GENERATOR_LOCATIONS BATTERY_LOCATIONS	Dict {gen_index: bus} Placeholder; overridden by solver
site_datacenter.py	—	Sweeps all 14 buses as datacenter locations, writes assets_dc files
solvers/uc.py	run_uc()	UC + ED (CVXPY/SCIP); fixed placement in, cost out
solvers/ed.py	run_ed()	ED only (CVXPY/HiGHS), continuous, no binary
solvers/siting_benders.py	run_siting_benders()	Benders decomposition siting (classical)
solvers/siting_mip.py	run_siting_mip()	Joint siting + UC MILP, single SCIP solve
solvers/quantum_siting.py	run_quantum_siting() build_proxy_cost_fn() build_butterfly_ansatz() run_vqa_qiskit() run_dwave_sa() compute_congestion_signal() evaluate_candidates()	Full hybrid pipeline: quantum sieve + classical refinement Builds $Q(u, s)$ proxy for COBYLA/VQA loop VQA circuit (butterfly ansatz, L layers) Qiskit Aer VQA: COBYLA + final sample D-Wave simulated annealing (BQM-based) Per-bus relief signal: PTDF $\times$ shadow prices Parallel classical UC/ED eval of quantum candidates

2 Network: IEEE 14-Bus

2.1 Bus Data

Bus	Type	Base Pd (MW)	Base Qd (MVar)	Notes
1	Slack (3)	0.0	0.0	Generator only
2	PV (2)	21.7	12.7	Generator + load
3	PV (2)	94.2	19.0	Generator + load
4	PQ (1)	47.8	-3.9	Load only; datacenter bus
5	PQ (1)	7.6	1.6	
6	PV (2)	11.2	7.5	Generator + load
7	PQ (1)	0.0	0.0	Transit bus
8	PV (2)	0.0	0.0	Generator only
9	PQ (1)	29.5	16.6	Shunt cap 19 MVar
10	PQ (1)	9.0	5.8	
11	PQ (1)	3.5	1.8	
12	PQ (1)	6.1	1.6	
13	PQ (1)	13.5	5.8	
14	PQ (1)	14.9	5.0	
Total base load		259.0		

2.2 Branch Data

Br#	From-To	r (pu)	x (pu)	b (pu)	Limit (MW)	Ratio	Note
0	1-2	0.01938	0.05917	0.0528	400	–	Strong backbone
1	1-5	0.05403	0.22304	0.0492	120	–	
2	2-3	0.04699	0.19797	0.0438	120	–	
3	2-4	0.05811	0.17632	0.0340	120	–	
4	2-5	0.05695	0.17388	0.0346	120	–	
5	3-4	0.06701	0.17103	0.0128	120	–	Local tie (unconstrained)
6	4-5	0.01335	0.04211	0.000	9999	–	
7	4-7	0.00000	0.20912	0.000	80	0.978	
8	4-9	0.00000	0.55618	0.000	40	0.969	
9	5-6	0.00000	0.25202	0.000	80	0.932	
10	6-11	0.09498	0.19890	0.000	80	–	Transformer; sole path to bus-6 cluster
11	6-12	0.12291	0.25581	0.000	60	–	
12	6-13	0.06615	0.13027	0.000	120	–	
13	7-8	0.00000	0.17615	0.000	80	–	
14	7-9	0.00000	0.11001	0.000	100	–	
15	9-10	0.03181	0.08450	0.000	200	–	
16	9-14	0.12711	0.27038	0.000	80	–	
17	10-11	0.08205	0.19207	0.000	80	–	
18	12-13	0.22092	0.19988	0.000	80	–	
19	13-14	0.17093	0.34802	0.000	60	–	

Key bottlenecks: Branch 8 (4–9, 40 MW) and Branch 9 (5–6, 80 MW) bind at peak and produce non-zero shadow prices with the datacenter at Bus 4.

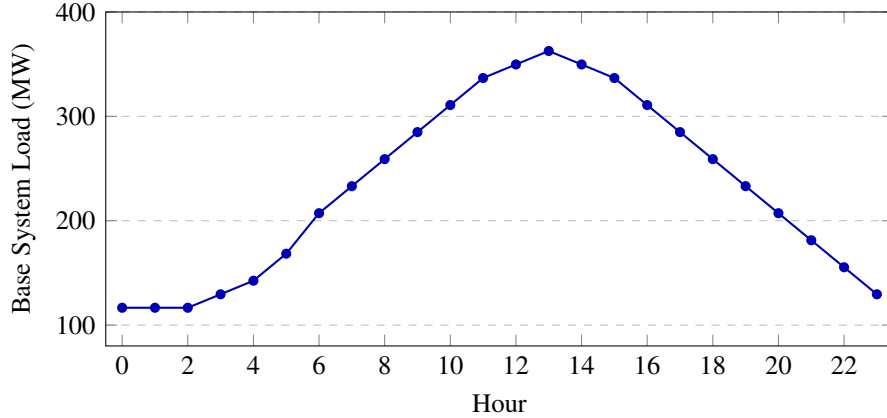
3 Generators

Five generators. Two cheap units (buses 1, 2 at \$20/MWh), three expensive units (buses 3, 6, 8 at \$40/MWh).

Name	Bus	Pmin (MW)	Pmax (MW)	cost_b (\$/MWh)	Startup (\$)
Gen 1	1	50	332	20	0
Gen 2	2	20	140	20	0
Gen 3	3	20	100	40	0
Gen 4	6	20	100	40	0
Gen 5	8	20	100	40	0
Total installed: 772 MW					

4 Load Profile

The 24-hour demand shape is defined in `ieee14.py` (`Case.factors`), a list of 24 hourly multipliers applied uniformly to all base bus loads. Source: synthetic daily shape scaled to the MATPOWER case14 base load of 259 MW. The 200 MW datacenter load at Bus 4 is added as a flat constant and is not multiplied by these factors. Peak system demand: 363 MW (base) + 200 MW (datacenter) = 563 MW at Hour 13.



5 Battery Specifications

Four identical batteries: **50 MW / 200 MWh, charge efficiency 90% (discharge lossless), initial SoC 200 MWh (fully charged)**. Defined in `assets.py` as `BATTERIES`.

6 Problem Formulation

6.1 Objective

The cost function in `solvers/uc.py` supports a quadratic-plus-linear form:

$$\min \sum_g \sum_{t=1}^{24} [a_g \cdot p_{g,t}^2 + b_g \cdot p_{g,t} + c_g \cdot u_{g,t} + sc_g \cdot v_{g,t}]$$

In this use case, $a_g = 0$ for all generators. The original MATPOWER case14 polynomial gencost data does include small quadratic terms, but this model simplifies to linear-only dispatch costs, consistent with the approach in Aboumradi et al. and necessary for keeping the QUBO proxy tractable on near-term hardware. No-load costs c_g and startup costs sc_g are also zero: startup events are tracked by $v_{g,t}$ (see below) but carry no cost in this formulation.

6.2 Decision Variables

- $p_{g,t} \geq 0$: generator output (MW)
- $u_{g,t} \in \{0, 1\}$: generator commitment (1 = online)
- $v_{g,t} \in \{0, 1\}$: startup indicator — equals 1 when generator g transitions from off ($u = 0$) at $t - 1$ to on ($u = 1$) at t , enforced by $v_{g,t} \geq u_{g,t} - u_{g,t-1}$. With $sc_g = 0$ it does not affect the objective but is retained for generality.
- $r_{b,t}^+ \geq 0$: battery charge (MW)
- $r_{b,t}^- \geq 0$: battery discharge (MW)
- $\text{SoC}_{b,t} \geq 0$: state of charge (MWh)
- $z_{b,t} \in \{0, 1\}$: direction flag (1 = charging) — prevents simultaneous charge and discharge

6.3 Constraints

Generator bounds:

$$P_g^{\min} \cdot u_{g,t} \leq p_{g,t} \leq P_g^{\max} \cdot u_{g,t}$$

Power balance (per hour):

$$\sum_g p_{g,t} + \sum_b (r_{b,t}^- - r_{b,t}^+) = \sum_i D_{i,t}$$

where $D_{i,t}$ is bus load at hour t (base load $\times$ factor, plus 200 MW flat at Bus 4).

DC line flow limits (per branch ℓ , per hour):

$$-\bar{f}_\ell \leq \sum_i \text{PTDF}_{\ell,i} \cdot \text{net}_{i,t} \leq \bar{f}_\ell$$

where $\text{net}_{i,t} = \text{injection}_{i,t} - D_{i,t}$.

Battery charge/discharge exclusion:

$$r_{b,t}^+ \leq P^{\text{bat}} \cdot z_{b,t}, \quad r_{b,t}^- \leq P^{\text{bat}} \cdot (1 - z_{b,t})$$

SoC bounds: $0 \leq \text{SoC}_{b,t} \leq 200$ MWh, with $\text{SoC}_{b,0} = 200$ MWh.

6.4 Siting Algorithm

The Benders decomposition (`siting_benders.py`) works as follows:

1. **Master problem** (SCIP MIP): selects which buses to place the 4 batteries on. Binary variables $x_{b,n}$; exactly one bus per battery; symmetry-breaking enforces ascending bus index for identical batteries.
2. **Subproblem** (`run_uc()`): given a fixed placement, solves the 24-hour UC/ED. Returns total cost Z .
3. **Cuts**: no-good cuts prevent re-visiting placements. L-shaped optimality cuts $\eta \geq Z \cdot (\sum_{(b,n) \in S^+} x_{b,n} - (M - 1))$ bound the master from below.
4. **Convergence**: master lower bound $\geq$ best known cost, or time limit.

An alternative joint MILP is in `siting_mip.py` (single SCIP solve, big-M linearisation of placement $\times$ dispatch).

7 Comparison

Run	Bus placement	24h cost
No batteries, no datacenter	—	\$209,229
No batteries, datacenter at Bus 4	—	\$228,429
Quantum (Qiskit Aer)	(2, 4, 6, 7)	\$199,804
Classical (SCIP)	(2, 4, 6, 7)	\$199,804

Quantum and classical solvers agree on both the optimal placement and cost. Quantum runtime: approximately 2.5 minutes on GPU. Classical solver: `run_siting_benders()` or `run_siting_mip()`.